

HelixTrack

HelixTrack

JIRA alternative pour le monde libre.

Résumé

HelixTrack est une alternative moderne, open source et complète à JIRA (et, via son extension Documents, à Confluence) — un système multiplateforme de gestion de projets et de suivi des problèmes, construit sur une architecture backend en microservices Go avec des clients natifs pour le web, le bureau et le mobile.

Description courte

Alternative open source à JIRA/Confluence. Un backend en microservices Go (« HelixTrack Core ») expose une API REST API unifiée pour le suivi des projets et des problèmes, ainsi qu'un espace de travail documentaire de type Confluence, servi à des clients natifs Web, Desktop, Android et iOS via HTTP/3 QUIC.

Description détaillée

HelixTrack est une plateforme open source de gestion de projets et de suivi des problèmes, conçue comme une alternative libre à JIRA et Confluence — un remplacement complet des deux outils auxquels la plupart des organisations d'ingénierie sont verrouillées, repensé sous la forme d'un logiciel que vous possédez et pouvez exécuter n'importe où. Son cœur est **HelixTrack Core**, un microservice REST API écrit en Go avec le framework Gin, offrant un suivi complet des problèmes, des tableaux agile/scrum, la gestion d'équipes et un moteur de permissions hiérarchiques dont l'implémentation peut être remplacée entre un moteur local intégré et un service accessible via HTTP — permettant ainsi au même modèle d'autorisation de s'adapter d'un simple ordinateur portable à un cluster distribué

sans modifier le code applicatif. Plutôt que d'étendre une surface REST sur des dizaines de routes, Core concentre tout sur un unique point d'entrée /do routé par actions, avec une enveloppe requête/réponse cohérente (action/jwt/object/data en entrée, errorCode/errorMessage/data en sortie) : tous les clients utilisent le même contrat minimal, et ajouter une fonctionnalité revient à ajouter une action, sans nécessiter une nouvelle URL à documenter, sécuriser et versionner. Core s'intègre avec des services découplés d'authentification, de permissions et de localisation, qui communiquent via HTTP/3 QUIC et peuvent s'exécuter sur des machines ou des clusters distincts, ou être désactivés entièrement dans des configurations de test. Les données sont stockées dans SQLite pour un développement sans configuration et dans PostgreSQL en production, chiffrées au repos avec SQLCipher (AES-256), garantissant ainsi que les données sensibles des projets sont protégées par défaut sur disque, plutôt que comme une réflexion a posteriori. Une extension **Documents V2** transforme le suivi en une plateforme de connaissances complète : un espace de travail de type Confluence avec des espaces, des pages, un contrôle de version, des modèles, une collaboration WebSocket en temps réel et des analyses — le wiki et le suivi des problèmes coexistant enfin derrière un seul backend, au lieu de deux produits assemblés artificiellement. Autour de Core gravitent plusieurs applications clientes : un client web Angular, un client desktop Tauri + Angular, des applications natives Android (Kotlin) et iOS (Swift), ainsi que des clients pour HarmonyOS et Aurora OS, et même un économiseur d'écran — tous communiquant avec le même backend et le découvrant automatiquement sur les réseaux locaux via des diffusions UDP, de sorte qu'un client fraîchement installé trouve son serveur sans configuration manuelle. Les applications clientes sont maintenues dans des dépôts privés distincts et ne sont présentées ici qu'au niveau du produit.

Contenu

Pourquoi nous l'avons créé

Pour offrir aux équipes une alternative véritablement ouverte et auto-hébergeable au duo JIRA + Confluence — « pour le monde libre » — sans verrouillage propriétaire, en réunissant suivi d'entreprise, documents et collaboration sous une seule licence open source.

Pourquoi c'est révolutionnaire

Il fusionne deux produits commerciaux lourds — le suivi des problèmes et la pile wiki/documents — en une plateforme unique, ouverte, performante et auto-hébergeable, tout en y ajoutant ce

que les acteurs établis n'ont jamais proposé : de véritables clients *natifs* multiplateformes (web, bureau, Android, iOS, ainsi que HarmonyOS et Aurora OS), tous pilotés par un même contrat backend. L'avancée majeure ? La propriété sans compromis. Une architecture HTTP/3 omniprésente, entièrement découplée en microservices, et un chiffrement SQLCipher AES-256 au repos offrent les performances et la sécurité habituellement réservées aux SaaS propriétaires, mais sur un système que vous hébergez vous-même — sans licences par poste, sans verrouillage propriétaire, sans que vos données quittent votre infrastructure. Les équipes bénéficient de l'expérience JIRA-plus-Confluence qu'elles connaissent déjà, sur leur propre matériel, sous une seule licence open source.

Ce qui est innovant

- Une interface unifiée basée sur les actions /do API — un seul point d'entrée, une seule enveloppe, routage par action. Les nouvelles fonctionnalités arrivent sous forme d'actions, et non de nouvelles URL, réduisant ainsi la surface d'attaque, le code client et la charge documentaire à un seul contrat partagé par toutes les plateformes.
- Le HTTP/3 QUIC comme *transport inter-services par défaut* — un réseau moderne à faible latence et résilient aux déconnexions entre services, intégré dès le premier jour, et non ajouté après coup.
- Un moteur de permissions interchangeable entre une implémentation locale en processus et un service accessible via HTTP, accompagné de services Authentification, Permissions et Localisation optionnels et déployables indépendamment — le même modèle d'autorisation, que vous exécutiez un seul processus ou un cluster.
- L'isolation des données multi-espaces via un drapeau `--space-root` : chaque projet dispose de sa propre base de données et de son propre stockage d'actifs isolés, séparant ainsi les locataires et les projets au niveau du stockage plutôt que par des filtres de requête.
- Le chiffrement SQLCipher AES-256 au repos — les données sensibles des projets sont protégées sur disque de manière transparente, par défaut.
- La découverte automatique client-serveur via diffusion UDP sur les réseaux locaux — un client trouve Core sans aucune configuration manuelle.
- Documents V2, une véritable « alternative à Confluence », avec édition parallèle en verrouillage optimiste, détection des conflits et historique complet des modifications — de vrais documents collaboratifs intégrés au même backend que le suivi.

Principaux défis techniques et solutions apportées

- **Six plateformes clientes, un backend, zéro dérive de contrat.** Maintenir des clients Web/Angular, Bureau/Tauri, Android/Kotlin, iOS/Swift, HarmonyOS et Aurora implique normalement six intégrations API divergentes qui se désynchronisent. Nous avons éliminé ce risque en faisant de l'interface action /do API et de son enveloppe fixe le *seul* contrat — chaque client le cible de manière identique — et en superposant une découverte de service par diffusion UDP pour que les clients localisent Core sur le réseau sans points de terminaison configurés manuellement.
- **Découpler les services sans payer un coût en latence.** Séparer Authentification, Permissions et Localisation en services déployables indépendamment ajoute normalement un saut réseau par appel. Nous avons adopté le HTTP/3 QUIC pour toutes les communications inter-services afin de maintenir ces sauts rapides et résilients aux déconnexions, et rendu chaque service exécutable indépendamment — voire entièrement désactivable dans les configurations de test — pour que le découplage reste un choix de déploiement, et non un coût fixe.
- **Une collaboration de niveau Confluence sans perte de données.** L'édition multi-auteurs en temps réel peut générer des conflits d'écriture. Documents V2 les résout grâce à des espaces/pages/versions sous verrouillage optimiste, une détection explicite des conflits, un historique complet des modifications pour revenir en arrière, et une synchronisation WebSocket en temps réel — une collaboration qui reste cohérente au lieu d'écraser silencieusement les modifications.
- **Chiffrement au repos sans sacrifier les performances.** Le chiffrement SQLCipher AES-256 protège les données sur disque, mais ajoute une surcharge par requête ; nous l'avons compensée par un cache multicouche (LRU en mémoire devant le Redis du service Localisation) pour que les chemins critiques, comme les recherches multilingues, restent rapides tout en gardant les données chiffrées.

Contenu

Pile technologique

- **Go + Gin** — choisis pour des services HTTP à haut débit et faible latence, avec un déploiement en binaire unique ; intègrent le REST API de Core, son middleware JWT/CORS, et le routeur d'actions /do qui sert de façade à l'ensemble du système.

- **HTTP/3 QUIC** — choisi comme protocole de transport entre Core et ses services d'Authentification, de Permissions et de Localisation, car la conception multiplexée et à migration de connexions de QUIC réduit la latence de queue et résiste aux liaisons instables, là où TCP se bloque.
- **PostgreSQL (prod) / SQLite (dev)** — un seul modèle relationnel pour le schéma étendu de suivi et de documents, exploité sur les deux moteurs : SQLite permet un développement local sans configuration et basé sur des fichiers, tandis que PostgreSQL prend le relais en production via un profil production dédié dans Compose.
- **SQLCipher (AES-256)** — choisi pour un chiffrement transparent au niveau de la base de données, protégeant les données sensibles des projets sans nécessiter de cryptographie au niveau applicatif ni modifier la manière dont les requêtes sont rédigées.
- **Redis** — utilisé comme couche de cache partagée derrière un LRU en mémoire dans le service de Localisation, offrant un cache à deux niveaux qui maintient les recherches multilingues rapides, même avec la surcharge du chiffrement en arrière-plan.
- **Uber Zap + Lumberjack** — choisis pour une journalisation structurée et légère en allocations, avec rotation intégrée, garantissant une observabilité en production sans croissance illimitée des logs.
- **golang-jwt / JWT** — choisis comme mécanisme d'authentification sans état ; le jeton signé est inclus dans le champ jwt de chaque enveloppe /do, assurant une authentification uniforme pour tous les clients.
- **Angular 19 (+ Material, RxJS)** — choisi pour un client navigateur réactif et basé sur des composants, avec un système de design Material mature intégré.
- **Tauri 2.0 + Rust + Angular** — choisis pour déployer une interface native légère en réutilisant l'UI de Angular dans une webview alimentée par Rust, évitant ainsi d'embarquer un runtime navigateur complet.
- **Kotlin (Android) / Swift + SwiftUI (iOS)** — choisis pour offrir aux utilisateurs mobiles des clients véritablement natifs et conformes aux idiomes de chaque plateforme, plutôt qu'une simple webview encapsulée.
- **Docker / Docker Compose (compatible Podman)** — choisis pour un déploiement conteneurisé reproductible, avec des vérifications /health intégrées, et une compatibilité Podman évitant toute dépendance à un démon ou un fournisseur spécifique.
- **Testify (Go) ; Cypress/Playwright/Karma+Jasmine (clients)** — choisis pour des tests automatisés en couches couvrant à la fois le contrat backend et les interfaces clients, en adéquation avec l'architecture un backend/multiples clients.

Statut et notes de transparence

- **Statut : bêta.** HelixTrack Core est un microservice REST API fonctionnel ; l'extension **Documents V2** est documentée comme étant achevée à environ 95 %, avec un problème connu de mappage des champs en base de données, et n'est donc pas présentée comme entièrement livrée.
- **Licence : à déterminer.** Le fichier CLAUDE.md indique une licence MIT, mais le fichier core/LICENSE de référence est en Apache 2.0 — cette divergence doit être résolue avant que la licence ne soit définitivement établie.
- Les chiffres de performance mentionnés dans le README du projet (par exemple, 50 000+ requêtes/seconde, temps de requête inférieurs à la milliseconde) sont des objectifs de conception et de marketing, et non des benchmarks publiés de manière indépendante ; ils sont donc omis des affirmations ci-dessus.
- Les applications clientes (Web, Desktop, Android, iOS, Aurora, HarmonyOS) sont hébergées dans des **dépôts privés** et ne sont décrites qu'au niveau du produit.

Niveau de priorité : Helix-primary et produit phare de la gamme Helix-Track — classé avant tout projet Server Factory.